



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sesión de Laboratorio 2:

PROBLEMA DEL LOGARITMO DISCRETO EN EC



SELECTED TOPICS OF CRYPTOGRAPHY

Profesora || Sandra Díaz Santiago
Grupo || 7CM1
Alumno || Areli Alejandra Guevara Badillo

24 de febrero de 2026

EJERCICIOS DE PROGRAMACIÓN

Se reutilizaron las funciones `sumaPuntos(P, Q, p)`, para la suma de dos puntos, y la función `suma2P(P, a, p)`, para doblar un punto, de la practica 1.

Ejercicio 1

Diseñe e implemente una función que reciba los parámetros a , b y p , es decir, los parámetros dados para una curva elíptica $E: y^2 = x^3 + ax + b \pmod{p}$, un punto $P \in E(a, b)$, y $k > 2$. Su función debe calcular y devolver el valor kP .

Código:

```
1 # Función para calcular kP en una EC sobre Zp
2 def kP(a, b, p, k, P):
3     if k == 1: return P
4     elif k % 2 == 0: return suma2P(kP(a, b, p, k // 2, P), a, p)
5     else: return sumaPuntos(P, kP(a, b, p, k - 1, P), p)
```

La función recibe los parámetros a , b , p que definen la curva y Z_p , a k que es un entero y un punto P en la curva representado como tupla (x, y, z) .

La función calcula $k \cdot P$ de forma recursiva utilizando tres casos:

1. Si $k == 1$, retorna directamente el punto P .
2. Si k es par, calcula recursivamente $(k/2) \cdot P$ y luego duplica el resultado usando `suma2P`.
3. Si k es impar, calcula recursivamente $(k-1) \cdot P$ y suma el punto P al resultado usando `sumaPuntos`.

Retorna el punto resultante de la multiplicación escalar kP .

Ejecución

```
Practica 2:
1. Calcular kP
2. Calcular logaritmo discreto
Seleccione una opción: 1
```

```
Ingresa a, b: 1, 9
Ingresa p: 1031
Ingresa k: 601
Ingresa x, y, z: 965, 626, 1
kP = 601 * (965, 626, 1) = (210, 936, 1)
```

```
Practica 2:
1. Calcular kP
2. Calcular logaritmo discreto
Seleccione una opción: 1
```

```
Ingresa a, b: 2,1
Ingresa p: 5
Ingresa k: 4
Ingresa x, y, z: 0,1,1
kP = 4 * (0, 1, 1) = (3, 2, 1)
```

Ejercicio 2

Diseñe e implemente una función para resolver el problema del logaritmo discreto en curvas elípticas, usando fuerza bruta. Su función debe recibir un número primo p , los parámetros para una curva elíptica $a, b \in \mathbb{Z}_p$, un punto generador $G \in E(a, b)$, y cualquier punto $P \in E(a, b)$.

Código:

```

1 # Función para calcular el logaritmo discreto en una EC sobre  $\mathbb{Z}_p$ 
2 def logartmoDiscretoEC(p, a, b, G, P, n=None):
3     limite = p + int(2 * math.sqrt(p)) + 2
4     if P == (0, 1, 0):
5         return 0
6
7     for k in range(1, limite):
8         if kP(a, b, p, k, G) == P:
9             return k
10    return -1

```

La función recibe como parámetro p que es el número primo que define el campo finito $\mathbb{Z}_p$, a, b que definen a la curva elíptica E , un punto G generador de la curva, representado como tupla (x, y, z) , y un punto P objetivo en la curva, representado como tupla (x, y, z) .

La función resuelve el problema del logaritmo discreto en curvas elípticas mediante fuerza bruta. Primero verifica si P es el punto al infinito $(0, 1, 0)$, en cuyo caso retorna 0 ya que $0G = O$. Luego, itera desde $k = 1$ hasta el límite superior del teorema de hasse ($p + 2\sqrt{p} + 2$), calculando kG en cada iteración usando la función kP . Compara el resultado con el punto P , y cuando encuentra coincidencia, retorna el valor de k . Este enfoque prueba todos los valores posibles de k secuencialmente hasta encontrar el que satisface la ecuación $P = kG$.

Retorna el entero k tal que $P = kG$, es decir, el logaritmo discreto de P en base G .

Ejecución

Use su función para encontrar k tal que $P = kG$, para cada una de las siguientes instancias del problema:

a) $p = 1031$, $a = 1$, $b = 9$, $G = (965 : 626 : 1)$, $P = (210 : 936 : 1)$

```

Ingrese p: 1031
Ingresa a, b: 1, 9
Ingresa x, y, z de G: (965, 626, 1)
Ingresa x, y, z de P: (210, 936, 1)
Logaritmo discreto: k = 601
Tiempo de ejecución: 0.0045 segundos

```

Sesión de Laboratorio 2: Problema del logaritmo discreto en EC

b) $p = 1048583$, $a = 1$, $b = 1$, $G = (531691 : 384179 : 1)$, $P = (1006535 : 412100 : 1)$

```
Ingrese p: 1048583
Ingresa a, b: 1, 1
Ingresa x, y, z de G: (531691, 384179, 1)
Ingresa x, y, z de P: (1006535, 412100, 1)
Logaritmo discreto: k = 1048613
Tiempo de ejecución: 27.6220 segundos
```

c) $p = 1073741827$, $a = 1$, $b = 11$, $G = (601392507 : 627288495 : 1)$, $P = (1002041819 : 647219641 : 1)$

```
Ingrese p: 1073741827
Ingresa a, b: 1, 11
Ingresa x, y, z de G: (601392507, 627288495, 1)
Ingresa x, y, z de P: (1002041819, 647219641, 1)
Logaritmo discreto: k = 1073641039
Tiempo de ejecución: 57964.8978 segundos
```

d) $p = 18446744073709551629$, $a = 1$, $b = 3$, $G = (3942083613116040372 : 17718673197222366791 : 1)$, $P = (13261599668560413002 : 18256791380151134939 : 1)$

Demasiado grande para ser calculado por fuerza bruta

e) $p = 2^{89} - 1$, $a = 2$, $b = 2$, $G = (382210962856070847682343483 : 378309917112278038228543048 : 1)$, $P = (506268261848751188944556411 : 327250297877515902632787381 : 1)$

Demasiado grande para ser calculado por fuerza bruta

f) $p = 2^{107} - 1$, $a = 7$, $b = 1$, $G = (125027947206083424147453383732008 : 42011786684287170889932732221307 : 1)$, $P = (61571535541812931520599016103231 : 58695302587616658417061315902087 : 1)$

Demasiado grande para ser calculado por fuerza bruta

PREGUNTAS

Responda las siguientes preguntas:

1. ¿Cuántas operaciones de suma de puntos y duplicación de puntos (aproximadamente) en el peor caso realiza su implementación para calcular kP ? ¿Cuándo ocurre el peor caso?

En el código, cada vez que se llama a $k // 2$, se está desplazando un bit a la derecha, lo cual indica que el ciclo de recursión se repetirá exactamente el número de bits que tenga k en binario.

Por lo que la implementación realiza aproximadamente $\log_2 k$ duplicaciones y, en el peor caso, $\log_2 k$ sumas. El peor caso ocurre cuando el valor de k tiene todos sus bits en 1, ya que esto hace que se ejecute una operación de suma adicional por cada bit procesado.

2. **Proporcione las características de la computadora que utiliza para ejecutar sus ejercicios de programación. Indique las especificaciones técnicas del CPU y la memoria.**

- **Sistema Operativo:** Windows 11
- **Procesador (CPU):** 12th Gen Intel(R) Core(TM) i5-12450H (2.50 GHz)
- **Arquitectura:** Sistema operativo de 64 bits, procesador x64
- **Núcleos:** 8 núcleos físicos / 12 procesadores lógicos
- **Memoria RAM:** 16.0 GB

3. **¿Cuánto tiempo tardó en su computadora resolver el problema del logaritmo discreto para cada instancia del punto 2 de la Sección 1?**

- a) 0.0045 segundos
- b) 31.2145 segundos
- c) 57964.8978 segundos

Para los siguientes casos se realizaron aproximaciones de tiempo debido a que nunca terminaron de ejecutarse, para esto se utilizó la siguiente formula:

$$\text{Tiempo en años} = \frac{p}{\text{velocidad en segundo} * \text{segundos en 1 año}}$$

d) 11 millones de años

- **Valor de p:** 18,446,744,073,709,551,629
- **Magnitud científica:** 1.84×10^{19} (es un número de 64 bits)
- $\frac{1.84 \times 10^{19}}{50,000 * 31,536,000} = 11,669,203 \text{ años}$

e) 390 billones de años

- **Valor de p:** $2^{89} - 1$
- **Magnitud científica:** 6.18×10^{26} (es un número de 89 bits)
- $\frac{6.18 \times 10^{26}}{50,000 * 31,536,000} = 3.9 \times 10^{14} \text{ años}$

f) 1.02×10^{20} años

- **Valor de p:** $2^{107} - 1$
- **Magnitud científica:** 1.62×10^{32} (es un número de 107 bits)
- $\frac{1.62 \times 10^{32}}{50,000 * 31,536,000} = 1.02 \times 10^{20} \text{ años}$

4. **Analice el siguiente pseudocódigo y escriba paso a paso qué hace este algoritmo para cada una de las siguientes entradas:**

- a) $k = 15$, $E = y^2 = x^3 + x + 3 \pmod{17}$, $P = (3, 4)$
- b) $k = 16$, $E = y^2 = x^3 + x + 3 \pmod{17}$, $P = (3, 4)$
- c) $k = 11$, $E = y^2 = x^3 + x + 3 \pmod{17}$, $P = (2, 8)$

Puede usar su programa para calcular la suma de puntos y la duplicación de puntos.

Algoritmo 1: Right-to-left binary method for point multiplication(k, P)

Input : $k = (k_{t-1}, \dots, k_1, k_0)$ binary representation of k ; $P \in \mathbb{E}(GF(p))$
Output: kP

```

1  $Q = \infty$ ;                                /* punto al infinito */
2 for  $i \leftarrow 0$  to  $t-1$  do
3   if  $k_i == 1$  then
4      $Q \leftarrow Q + P$ ;                    /* point addition */
5   end
6    $P \leftarrow 2P$ ;                        /* point doubling */
7 end
8 return  $Q$ 
```

	k	P	Q	i	k_i	$k_i = 1?$	Q	P	return Q
a)	15 1 1 1 1 $k_3 k_2 k_1 k_0$	(3, 4)	∞	0	1	✓	(3, 4)	(2, 8)	(2, 9)
				1	1	✓	(11, 11)	(12, 3)	
				2	1	✓	(7, 8)	(8, 8)	
				3	1	✓	(2, 9)	(3, 13)	
b)	16 1 0 0 0 0 $k_4 k_3 k_2 k_1 k_0$	(3, 4)	∞	0	0	✗	∞	(2, 8)	(3, 13)
				1	0	✗	∞	(12, 3)	
				2	0	✗	∞	(8, 8)	
				3	0	✗	∞	(3, 13)	
				4	1	✓	(3, 13)	(2, 9)	
c)	11 1 0 1 1 $k_3 k_2 k_1 k_0$	(2, 8)	∞	0	1	✓	(2, 8)	(12, 3)	(6, 2)
				1	1	✓	(16, 16)	(8, 8)	
				2	0	✗	(16, 16)	(3, 13)	
				3	1	✓	(6, 2)	(2, 9)	

¿Es este algoritmo diferente del algoritmo que implementó para el punto 1 de la Sección 1?

Sí, son diferentes en la forma en que trabajan con el número k :

- Mi algoritmo: Divide el problema en partes más pequeñas. Va desde los bits más grandes hacia los más pequeños (de izquierda a derecha).
- El Algoritmo 1: Es un ciclo simple que va de derecha a izquierda. Revisa los bits de k uno por uno, empezando por el final, y va acumulando el resultado en cada paso.

¿Cuál es mejor, el suyo o este algoritmo anterior? ¿Por qué?

El Algoritmo 1 es mejor para aplicaciones reales por tres razones:

- Usa menos memoria: Mi código recursivo guarda muchas "tareas pendientes" en la memoria de la computadora. Si el número k es gigantesco, la computadora se puede quedar sin memoria. El Algoritmo 1 no tiene este problema porque usa un ciclo simple.
- Es más estable: Al no usar recursión, el programa es más predecible y difícil de romper.
- Es más rápido con números grandes: Aunque ambos hacen casi la misma cantidad de cuentas, el Algoritmo 1 ahorra el tiempo que la computadora gasta entrando y saliendo de funciones constantemente.